MCS-220: Web Technologies (June 2010 – June 2023) – Questions and Answers

This document compiles selected questions from the IGNOU MCS-220 Web Technologies exams (June 2010 through June 2023) with detailed, academic-style answers. Each question is followed by its comprehensive answer, including clear explanations, diagrams (described in text), and code examples in Java web technologies (Servlets, JSP, Spring, Hibernate, etc.). Citations to authoritative sources are provided to support key points.

June 2022 Term-End Examination

Question 1(a) – Web Server vs. Web Container

Q: *Differentiate between a web server and a web container (application server) in the context of Java web applications.*
A: A **web server** (e.g. Apache HTTPD, Nginx, Microsoft IIS) is a program that listens for HTTP requests and serves static content (HTML, CSS, images) to clients. It handles basic HTTP protocol operations. In contrast, a **web container** (also called a servlet container or application server, e.g. Apache Tomcat, Jetty) is a component that runs within a web server (or standalone) and manages Java servlet and JSP execution. The web container is responsible for the **servlet lifecycle** (loading, initializing, servicing, and destroying servlets) and dispatching requests to the appropriate servlet or JSP [1] [2].

For example, when a client requests a dynamic page, the web server forwards the request to the web (servlet) container. The container then creates (if not already loaded) the servlet class, calls its `init()` method, and invokes `service(request, response)`, which in turn calls `doGet()` or `doPost()`. The web container handles `HttpSession`, security, and JNDI lookups. In summary, the web server handles static content and basic HTTP, while the web container (servlet engine) provides the runtime for Java servlets/JSPs and enterprise Java features [1].

Question 1(b) – Servlet Architecture and Example

Q: *Explain the architecture of a Java Servlet. Illustrate with a servlet example that handles HTTP GET and POST.*
A: Java Servlets follow a request–response architecture. The **client** (usually a browser) sends an HTTP request to the server. The **servlet container** receives the request and finds the matching servlet based on the URL mapping. The container instantiates (or uses an existing instance of) the servlet and calls its life-cycle methods. Specifically, after `init()`, the container invokes the servlet's `service(request,response)` method on each request. The `service` method examines the HTTP method (GET, POST, etc.) and calls `doGet` or `doPost` accordingly. The servlet processes the request and writes output to the `HttpServletResponse`, which the container then sends back to the client.

Servlet Life Cycle: Loading → `init()` → `service()` (multiple calls for requests) → `destroy()` [2] [3]. The container manages this life cycle. For example, the GeeksforGeeks tutorial explains that servlets go through *loading*, *initialization*, *request handling*, and *destruction* phases [2] [3].

Example Servlet: Below is a simple Java servlet that handles both GET and POST requests using `doGet()` and `doPost()`. It reads a parameter `name` from the request and writes a greeting.

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class HelloServlet extends HttpServlet {
    // Called by the server (once) to initialize the servlet
    public void init() throws ServletException {
        // initialization code, e.g. database connections
    }

    // Handle GET requests
    protected void doGet(HttpServletRequest request, HttpServletResponse
response)
            throws ServletException, IOException {
        String name = request.getParameter("name");
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<html><body>");
        if (name != null) {
            out.println("<h1>Hello, " + name + "!</h1>");
        } else {
            out.println("<h1>Hello, World!</h1>");
        }
        out.println("</body></html>");
    }

    // Handle POST requests (delegates to doGet)
    protected void doPost(HttpServletRequest request, HttpServletResponse
response)
            throws ServletException, IOException {
        doGet(request, response);
    }

    // Called by the server to destroy the servlet before shutdown
    public void destroy() {
        // cleanup code
    }
}
```

In this example, the servlet container will call `init()` once, and then for each request call `doGet()` or `doPost()`. When the server is shut down or the servlet is unloaded, `destroy()` is called. This illustrates the servlet request-response flow and life-cycle [2] [3].

Question 1(c) – JSP Scripting Elements

Q: *What are JSP scripting elements? Explain with examples.*
A: JSP scripting elements allow embedding Java code and declarations within a JSP page. They must be enclosed in special tags and are processed by the JSP engine. The three main scripting elements are [4]:

- **Scriptlet** (`<% ... %>`): Contains Java code that is inserted into the servlet's `service()` method.
- **Expression** (`<%= ... %>`): Evaluates a Java expression and inserts its result into the HTML response (equivalent to `out.println(...)`).
- **Declaration** (`<%! ... %>`): Declares fields or methods in the generated servlet class (outside `service()`).

For example, in a JSP:

```
<%@ page import="java.util.Date" %>
<html>
  <body>
    <%-- Scriptlet: execute Java code --%>
    <%
       int count = 5;
    %>
    <h2>Count is: <%= count %></h2>  <!-- Expression: prints value of count
-->
    <%!  // Declaration: define a method
       private String getGreeting() {
           return "Welcome!";
       }
    %>
    <p><%= getGreeting() %></p>
  </body>
</html>
```

In this code, `<% int count = 5; %>` is a scriptlet where Java code executes. `<%= count %>` is an expression that outputs the value of `count`. The declaration block `<%! ... %>` adds the method `getGreeting()` to the servlet class.

JSP scripting elements enable dynamic content generation by mixing Java code and HTML. They are an essential feature of JSP as documented by W3Schools [4].

Question 1(d) – Spring Boot Properties

Q: *Explain the use of* `application.properties` *(or* `application.yml`*) in Spring Boot.*
A: Spring Boot uses `application.properties` or `application.yml` files to externalize configuration. These files (located in `src/main/resources`) allow setting various application settings without hard-coding values. Common properties include server port, database connection details, and custom application settings. For example, one might set `server.port=8081` to run on port 8081 instead of 8080.

Properties from these files are automatically loaded and made available in Spring's `Environment`. Beans can inject values using `@Value` or bind to `@ConfigurationProperties`. For instance:

```
spring.datasource.url=jdbc:mysql://localhost/mydb
spring.datasource.username=root
spring.datasource.password=secret
```

These properties configure the Spring DataSource. The Spring Boot documentation explains that this external configuration enables flexibility and overrides (e.g. via profiles) [5].

Question 2(a) – Design Patterns in Web Applications

Q: *What are software design patterns? State their advantages and give an example (e.g. Singleton or Factory pattern).*
A: Software design patterns are **reusable solutions** to common design problems. They are not code templates, but descriptions of how to solve problems that can be used in many different situations [3] [5]. Patterns help architects and developers avoid reinventing the wheel by providing proven approaches. Advantages include improved code reusability, maintainability, and clarity of design intent.

For example, the **Singleton pattern** ensures a class has only one instance and provides a global access point. In a web context, a singleton can manage shared resources (e.g. a connection pool). In Java:

```
public class AppConfig {
    private static AppConfig instance;
    private AppConfig() { /* private constructor */ }
    public static synchronized AppConfig getInstance() {
        if (instance == null) {
            instance = new AppConfig();
        }
        return instance;
    }
    // additional config methods...
}
```

This `AppConfig` class can only have one instance at runtime.

Another example is the **Factory pattern**, which encapsulates object creation. For instance, a `DAOFactory` might create DAO objects for different databases. Clients call a factory method instead of using `new` directly, allowing the factory to decide which class to instantiate at runtime.

These are formalized in resources such as the book *Design Patterns: Elements of Reusable Object-Oriented Software* [3]. Using design patterns leads to cleaner, more maintainable code by applying best practices.

Question 2(b) – Hibernate Configuration with Annotations

Q: *How do you configure Hibernate in a Spring application using Java annotations? Provide an example entity.*
A: In Spring, Hibernate (JPA) configuration can be done with annotations without XML. Key steps: 1. **Entity class with annotations:** Use `@Entity`, `@Table`, `@Id`, etc., on your model class.

2. **Spring configuration:** Enable JPA repositories (`@EnableJpaRepositories`), and define a `LocalContainerEntityManagerFactoryBean` and `DataSource` bean (often automatically configured by Spring Boot if you include `spring-boot-starter-data-jpa` and set datasource properties).

Example Entity:

```
import javax.persistence.*;

@Entity
@Table(name="employees")
public class Employee {
    @Id @GeneratedValue(strategy=GenerationType.IDENTITY)
    private Long id;

    @Column(name="first_name", length=50, nullable=false)
    private String firstName;

    @Column(name="last_name", length=50, nullable=false)
    private String lastName;

    // Constructors, getters, setters
    public Employee() {}
    public Employee(String firstName, String lastName) {
        this.firstName = firstName; this.lastName = lastName;
    }
    // Getters and setters...
}
```

Then, in a Spring Boot app, include in `application.properties` the datasource and JPA properties (e.g. `spring.jpa.hibernate.ddl-auto=update`). Spring Boot auto-configures the `EntityManagerFactory` and `DataSource` from these settings. The `@Entity` annotation tells Hibernate to map this class to the `employees` table.

This annotation-based setup eliminates XML mappings. Entities are scanned by Spring Boot (via package scanning), and Hibernate uses the annotated metadata to generate SQL and manage persistence [6].

Question 3(a) – Spring MVC and Flow Diagram

Q: *Describe the Spring MVC workflow. Include a diagram (in text) illustrating the flow from client request to response.*
A: Spring MVC is based on the Model-View-Controller pattern. The workflow is as follows:

1. **Client Request:** Browser sends an HTTP request to the server.
2. **DispatcherServlet:** The central **front controller** is Spring's `DispatcherServlet`, configured in `web.xml` or via Java config. All requests go through it.

3. **Handler Mapping:** `DispatcherServlet` consults `HandlerMapping` to decide which controller method (handler) should process the request (based on URL and annotations like `@RequestMapping`).
4. **Controller Invocation:** The matched controller (`@Controller` annotated bean) method is called. It can populate a `Model` (typically returning a `ModelAndView` or view name and model data).
5. **View Resolution:** After the controller returns, `DispatcherServlet` uses a `ViewResolver` to select the appropriate view (JSP, Thymeleaf template, etc.) and renders it, merging the model data.
6. **Response:** The generated HTML (or JSON) is sent back in the `HttpServletResponse` to the client.

Text Diagram:

```
Client HTTP Request
    ↓
[DispatcherServlet]
    ↓
HandlerMapping (maps URL to Controller)
    ↓
[Controller Method @RequestMapping] → (calls business service, populates
Model)
    ↓
HandlerAdapter (invokes method, returns ModelAndView)
    ↓
ViewResolver (selects view template)
    ↓
View (renders model data to HTML)
    ↓
HTTP Response back to Client
```

This separation allows clear structure: the **Model** holds data, the **View** renders it, and the **Controller** processes input and orchestrates the model-view mapping.

This flow is detailed in Spring's documentation [5] [7], which explains the role of `DispatcherServlet` and supporting components in handling web requests.

Question 3(b) – Struts 2 Architecture and Flow

Q: *Explain how Struts 2 processes a request (include core components like FilterDispatcher, Action, ValueStack, etc.).*
A: Struts 2 is an MVC framework where the **controller** is implemented by a servlet filter (`FilterDispatcher` in older versions, now `StrutsPrepareAndExecuteFilter`). The flow is:

1. **Request Intercepted:** The `FilterDispatcher` intercepts the HTTP request (configured in `web.xml`), acting as the front controller [8].
2. **Action Mapping:** It looks up the action configuration (from `struts.xml` or annotations) to find the `Action` class that should handle this URL.

3. **Interceptors:** A stack of **interceptors** executes (e.g., for validation, file upload handling, etc.) before and after action execution.
4. **Action Execution:** Struts creates (or reuses) the **Action** object (a simple POJO) and calls its `execute()` method. The action may use a **ModelDriven** object for data.
5. **ValueStack/OGNL:** Struts pushes objects onto the **ValueStack** (action, models, temporary objects). JSP/Velocity views use OGNL expressions to access these. OGNL stands for Object-Graph Navigation Language, an expression language for retrieving values from the ValueStack [9]. For example, `<s:property value="name"/>` retrieves `name` from the action or model on the stack.
6. **View (Result):** After action returns a result string (e.g. "success"), Struts uses a **Result** (often a JSP) to render the response, passing data from the ValueStack to the view.

Textual Flow Diagram:

```
HTTP Request
   ↓ (struts2 filter)
FilterDispatcher (Front Controller)
   ↓ (determine Action)
[Action] (POJO)
   ↓ (interceptor stack pre-processing)
Interceptors (validation, params, etc.)
   ↓
Action.execute()
   ↓ (populate Model & ValueStack)
[ValueStack] (holds action and data)
   ↓
Interceptors (post-processing)
   ↓
Return result (e.g. "success")
   ↓
Result (View: JSP/Velocity) rendered, using OGNL to display data from
ValueStack
   ↓
HTTP Response
```

Here, the **ValueStack** is a stack of objects (action, models, etc.) that the view can access [10] [9]. OGNL allows concise property access. The JRebel article on Struts confirms the role of the ActionServlet (FilterDispatcher) as controller and the MVC architecture [8]. The **core components** of Struts 2 include the FilterDispatcher, Action classes, Interceptors, ValueStack, and Result pages.

Question 4(a) – Using JDBC in JSP

Q: *Write a JSP snippet (or program) to connect to a database using JDBC and display query results.*
A: In a JSP, one can use JSP scriptlets to perform JDBC operations (though best practice is to use Servlets/DAO; but for demonstration we show JSP code). For example, suppose we have a MySQL `employees` table. Below is a simplified JSP snippet that connects via JDBC, executes a SELECT, and displays the results:

```jsp
<%@ page import="java.sql.*" %>
<html>
<body>
    <h2>Employee List</h2>
    <%
        String url = "jdbc:mysql://localhost:3306/mydb";
        String user = "root";
        String pass = "secret";
        Connection conn = null;
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            conn = DriverManager.getConnection(url, user, pass);
            Statement stmt = conn.createStatement();
            String query = "SELECT id, first_name, last_name FROM employees";
            ResultSet rs = stmt.executeQuery(query);
            out.println("<table border='1'><tr><th>ID</th><th>Name</th></
tr>");
            while (rs.next()) {
                int id = rs.getInt("id");
                String name = rs.getString("first_name") + " " +
rs.getString("last_name");
                out.println("<tr><td>" + id + "</td><td>" + name + "</td></
tr>");
            }
            out.println("</table>");
        } catch (Exception e) {
            out.println("Database error: " + e.getMessage());
        } finally {
            if (conn != null) conn.close();
        }
    %>
</body>
</html>
```

This JSP uses a scriptlet to load the JDBC driver, establish a connection, execute a query, and iterate over the `ResultSet`. Each row is printed as an HTML table row. **Note:** In production, embedding JDBC in JSP is discouraged; one would use a servlet or data access layer. But this illustrates the concept.

Question 4(b) – Hibernate `load()` vs `get()`

Q: *Explain the difference between Hibernate's* `load()` *and* `get()` *methods.*
A: In Hibernate's `Session` interface:

- `get(Class, id)`: Immediately hits the database and returns the object (or `null` if not found). It returns the real object. Use when you need to access the object right away.
- `load(Class, id)`: Returns a proxy without hitting the database initially. The database is queried only when a property of the object is accessed. If no row exists, `load()` throws an exception (`ObjectNotFoundException`) when accessed. Use when you are sure the object exists and you want lazy loading.

In short, `get()` is eager fetch (returns `null` if no record) whereas `load()` is lazy (throws exception if no record when accessed). Both are described in Hibernate documentation. These behaviors are important when you want to optimize performance or handle missing data carefully.

Question 5(a) – JCA

Q: *Write short notes on: (a) JCA (Java Connector Architecture).*
A: JCA (Java Connector Architecture) is a Java EE solution for connecting application servers to Enterprise Information Systems (EIS) such as ERP, mainframes, databases, and legacy systems. It defines a standard set of System-level contracts (interfaces) between the EIS and application servers, enabling resource adapters that allow Java EE applications (EJBs, servlets, etc.) to access non-Java systems. JCA provides **connection pooling, transaction management, and security** integration for resource adapters [11] [12]. Essentially, JCA makes it easier to integrate enterprise systems into a Java EE application server environment by providing a pluggable architecture.

Question 5(b) – JSTL

Q: *Write short notes on: (b) JSTL.*
A: JSTL (JSP Standard Tag Library) is a collection of JSP tags that encapsulate common tasks. It provides tags for iteration (`<c:forEach>`), conditional logic (`<c:if>`, `<c:choose>`), internationalization (`<fmt:message>`), SQL (in development), XML, and more. By using JSTL, developers can avoid writing scriptlets (`<% ... %>`) in JSPs and instead use clean, semantic tags for logic and data access in the view. For example, `<c:forEach var="user" items="${userList}"> ... </c:forEach>` iterates over a collection. JSTL improves readability and maintainability of JSP pages. The W3Schools JSP guide and standard references list all available JSTL tags.

Question 5(c) – JSP Life Cycle

Q: *Write short notes on: (c) JSP Life Cycle.*
A: The **JSP Life Cycle** is similar to a servlet's but includes a compilation step. The phases are [13] [14]:

1. **Translation/Compilation:** The JSP file is translated into a Java servlet source (.java) and then compiled into a `.class`. This happens the first time the JSP is requested or if it has changed.
2. **Initialization (`jspInit()`):** The servlet's `init()`-like method `jspInit()` is called once. Any one-time setup (e.g. opening resources) occurs here.
3. **Execution (`_jspService()`):** For each request, the servlet's service method (`_jspService`) is invoked, processing JSP tags and scriptlets to generate the response.
4. **Destroy (`jspDestroy()`):** When the server unloads the JSP (e.g. on shutdown), `jspDestroy()` is called to free resources.

Tutorialspoint describes these four phases (Compilation, Initialization, Execution, Cleanup) and notes that JSP compilation includes parsing the JSP and converting to a servlet [13] [14]. Understanding this lifecycle helps in debugging and optimizing JSPs.

June 2023 Term-End Examination

Question 1(a) – Web Server vs. Servlet Container

Q: *Differentiate between a web server and a web container (servlet container).*
A: (Answer similar to Q1a of June 2022) A **web server** handles HTTP requests and serves static content,

while a **web container** (servlet container) runs within a web server (or is embedded) and manages Java servlets and JSPs. The container handles servlet lifecycle, request dispatching, sessions, etc., whereas the web server itself does not process servlets.

Question 1(b) – GenericServlet vs HttpServlet

Q: *Compare* `GenericServlet` *with* `HttpServlet`.
A: `GenericServlet` and `HttpServlet` are both base classes in the Java Servlet API:

- `GenericServlet` is a protocol-independent servlet (in `javax.servlet` package). It requires overriding the `service(ServletRequest, ServletResponse)` method. It is rarely used directly.
- `HttpServlet` (in `javax.servlet.http`) extends `GenericServlet` and is HTTP-specific. It provides methods `doGet()`, `doPost()`, etc. Most web applications use `HttpServlet` because it is tailored to HTTP. For example:

```
public class MyServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse
res) { /*...*/ }
    protected void doPost(HttpServletRequest req, HttpServletResponse
res) { /*...*/ }
}
```

with `HttpServlet`, you implement methods for each HTTP verb instead of handling `ServletRequest` manually. In summary, `HttpServlet` simplifies writing servlets for HTTP, whereas `GenericServlet` is more general-purpose [1] [2].

Question 1(c) – Session Management with Cookies

Q: *What is session management? Write a servlet program to manage user sessions using cookies.*
A: Session management allows a web application to keep state across multiple requests from the same client. Because HTTP is stateless, sessions provide a way to associate multiple requests with a single user. In Java, sessions are typically managed using `HttpSession`. Cookies (or URL rewriting) are used to track the client.

Example Servlet:

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class LoginServlet extends HttpServlet {
    protected void doPost(HttpServletRequest request, HttpServletResponse
response)
            throws ServletException, IOException {
        String user = request.getParameter("username");
        String pass = request.getParameter("password");
        // (Authenticate user here; assume success)
        HttpSession session = request.getSession();
```

```
        session.setAttribute("user", user);
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<html><body>");
        out.println("Welcome " + user);
        out.println("<br><a href='welcome'>Go to Welcome Page</a>");
        out.println("</body></html>");
    }
}

public class WelcomeServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse
response)
            throws ServletException, IOException {
        // Use existing session, do not create new
        HttpSession session = request.getSession(false);
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        if (session != null && session.getAttribute("user") != null) {
            String user = (String) session.getAttribute("user");
            out.println("<html><body>");
            out.println("Welcome back, " + user + "!");
            out.println("</body></html>");
        } else {
            out.println("No active session. Please <a
href='login.html'>login</a>.");
        }
    }
}
```

In `LoginServlet`, after authenticating, we call `request.getSession()` to create an `HttpSession` and store the username as an attribute. A session cookie (`JSESSIONID`) is sent to the browser. In `WelcomeServlet`, `getSession(false)` retrieves the existing session (without creating a new one). If the session exists, we retrieve `user` from it.

The BeginnersBook tutorial describes using `session.setAttribute()` and `getAttribute()` to store and retrieve user data [12] [15]. This demonstrates simple session management with cookies in servlets.

Question 1(d) – Spring Boot DevTools

Q: *List some features of Spring Boot DevTools.*
A: Spring Boot DevTools is a development-only module that improves developer productivity. Key features include: - **Automatic Restart:** It restarts the application on classpath changes, speeding up development.
- **LiveReload:** If a compatible browser extension is installed, the page reloads when resources change.
- **Global Settings:** Supports global default properties (for working directory).
- **Remote Debugging:** Allows quick restart over JMX/HTTP.
- **Configurations:** By default, disables template caching (for Thymeleaf, etc.) to reflect changes immediately.

These features help developers see code changes without manual restarts. The official Spring Boot docs detail these DevTools features.

Question 2(a) – JDBC Connection Pooling in Spring

Q: *Explain how to configure connection pooling in Spring (e.g. using* `DataSource`*).*
A: In Spring (or Spring Boot), connection pooling is usually configured via a `DataSource` bean. Spring Boot auto-configures a connection pool if you include a supported pool library (HikariCP by default). In a traditional Spring app, one can define a `DataSource` bean (e.g., `org.apache.commons.dbcp2.BasicDataSource` or `com.zaxxer.hikari.HikariDataSource`) in XML or Java config:

```
@Bean
public DataSource dataSource() {
    HikariDataSource ds = new HikariDataSource();
    ds.setJdbcUrl("jdbc:mysql://localhost/mydb");
    ds.setUsername("root");
    ds.setPassword("secret");
    ds.setMaximumPoolSize(10);
    return ds;
}
```

This `DataSource` manages a pool of connections. Spring's `JdbcTemplate` or `EntityManager` will use this pooled datasource. By using a pool, connections are reused rather than opened/closed on each request, improving performance under load.

Question 2(b) – Spring MVC Configuration Flow

Q: *Describe the process of configuring Spring MVC via Java-based configuration (no XML).*
A: To configure Spring MVC without XML, one typically:

1. **WebInitializer:** Create a class implementing `WebApplicationInitializer` (or extend `AbstractAnnotationConfigDispatcherServletInitializer`) to replace `web.xml`. This bootstraps the Spring context.
2. **Configuration Class:** Create a `@Configuration` class annotated with `@EnableWebMvc`. In it, define component scanning (`@ComponentScan`) for controllers and configure view resolvers, resource handlers, etc. For example:

   ```
   @Configuration
   @EnableWebMvc
   @ComponentScan("com.example.app")
   public class WebConfig implements WebMvcConfigurer {
       @Bean
       public ViewResolver viewResolver() {
           InternalResourceViewResolver vr = new
   InternalResourceViewResolver();
           vr.setPrefix("/WEB-INF/views/");
           vr.setSuffix(".jsp");
           return vr;
   ```

```
    }
    // Additional configuration (static resources, message converters,
etc.)
}
```

3. **DispatcherServlet Registration:** In `WebApplicationInitializer`, register `DispatcherServlet` with the `WebConfig` context. For example:

```
public class AppInitializer extends
AbstractAnnotationConfigDispatcherServletInitializer {
    @Override
    protected Class<?>[] getRootConfigClasses() {
        return null;
    }
    @Override
    protected Class<?>[] getServletConfigClasses() {
        return new Class[] { WebConfig.class };
    }
    @Override
    protected String[] getServletMappings() {
        return new String[] { "/" };
    }
}
```

4. **Controllers:** Use `@Controller` and `@RequestMapping` for handlers.

This Java config eliminates XML. The flow is: The servlet container (Tomcat) sees `AppInitializer` which registers a `DispatcherServlet` using `WebConfig`. The annotations in `WebConfig` (`@EnableWebMvc`, `@ComponentScan`) wire up Spring MVC infrastructure [5]. This is the standard approach in modern Spring apps (detailed in Spring docs).

Question 3(a) – Servlet Collaboration (RequestDispatcher vs sendRedirect)

Q: *What is servlet collaboration? How do* `RequestDispatcher` *(forward/include) and* `sendRedirect()` *differ?*
A: Servlet collaboration is when one servlet or JSP hands off control to another resource (servlet, JSP, or static content) within the same web application. There are two main ways:

- **RequestDispatcher.forward(request,response):** Server-side forward. The original request is forwarded to another resource on the server. The browser's URL remains unchanged. No new request is made by the client. Attributes in the `request` and `session` are preserved.
- **RequestDispatcher.include(request,response):** The content of another resource is included in the response of the current servlet. Control then returns to the original servlet, and additional content can be appended.
- **HttpServletResponse.sendRedirect(url):** Client-side redirect. The server sends a 302 response with a new URL. The client's browser then issues a new request to that URL. This changes the URL in the browser. Session can be maintained (via cookie or URL rewriting), but request attributes do *not* carry over to the new request.

Example:

```
// In servlet1:
if (validUser) {
    RequestDispatcher rd = request.getRequestDispatcher("welcome.jsp");
    rd.forward(request, response);
} else {
    response.sendRedirect("error.html");
}
```

Here, a successful login forwards to `welcome.jsp` internally, while failure redirects the client to `error.html`. The GeeksforGeeks article on servlet collaboration explains this process [16] [17]. Forwarding is more efficient (no round trip) and keeps request data, whereas redirect triggers a new client request.

Question 3(b) – JSP Life Cycle (different part)

Q: *What is the life cycle of a JSP? Compare it to the servlet life cycle.*
A: (Answer similar to earlier JSP life cycle) A JSP's life cycle is: **Translation** (JSP → servlet code) and **Compilation** (servlet `.java` to `.class`), then **Initialization** (`jspInit()`), repeated **Request Handling** (`_jspService()` for each request), and finally **Destroy** (`jspDestroy()`). This is akin to a servlet's life cycle (load → init → service → destroy) with the extra initial compilation step [13]. In essence, once a JSP is converted to a servlet, it behaves like any other servlet, but the conversion and translation step is unique to JSP.

Question 4(a) – JDBC in JSP (alternative example)

Q: *Write JSP code to connect to a database and retrieve data using JDBC.*
A: (Similar to Q4a above) Use JSP scriptlets to load the driver, get a connection, execute a query, and display results. For brevity, see the example in Q4(a) above. That example demonstrates the key concepts of `DriverManager.getConnection(...)`, `Statement.executeQuery(...)`, and iterating `ResultSet`.

Question 4(b) – Hibernate Associations (One-to-Many)

Q: *Explain how to map a one-to-many relationship in Hibernate using annotations.*
A: In Hibernate (JPA), a one-to-many relationship means one entity is related to a collection of another entity. For example, one `Department` has many `Employee`s. With annotations:

```
@Entity
public class Department {
    @Id @GeneratedValue
    private Long id;
    private String name;

    @OneToMany(mappedBy="department", cascade=CascadeType.ALL,
fetch=FetchType.LAZY)
    private List<Employee> employees;
    // getters/setters
```

```
}

@Entity
public class Employee {
    @Id @GeneratedValue
    private Long id;
    private String name;

    @ManyToOne
    @JoinColumn(name="department_id")
    private Department department;
    // getters/setters
}
```

In `Department`, `@OneToMany` on `employees` is linked by `mappedBy="department"` (the field in `Employee`). The `cascade` and `fetch` settings control persistence cascade and loading. In `Employee`, `@ManyToOne` and `@JoinColumn` create the foreign key. This bidirectional mapping ensures that a department's list contains its employees and each employee knows its department. Hibernate/JPA handles the underlying SQL join. An official JPA tutorial confirms these annotations for one-to-many mappings.

Question 5(a) – JSP Implicit Objects

Q: *Explain any four JSP implicit objects with examples.*
A: JSP provides several built-in implicit objects (accessible without declaration). Four common ones are [18] [19]: - `request` (`HttpServletRequest`): Represents the client's request. Can be used to get parameters (`request.getParameter("name")`) or attributes.
- `response` (`HttpServletResponse`): Represents the response to send to client. Has methods like `response.setContentType()` or `response.getWriter()`.
- `out` (`JspWriter`): Used to write to the JSP output stream (e.g., `out.println("Hello");`).
- `session` (`HttpSession`): Represents the user's session; you can call `session.getAttribute("user")` or `session.setAttribute(...)`.

For example, in JSP:

```
<%
    String name = request.getParameter("name");
    Integer visits = (Integer) session.getAttribute("visits");
    if (visits == null) visits = 0;
    visits++;
    session.setAttribute("visits", visits);
    out.println("Hello, " + name + "! This is your visit #" + visits);
%>
```

Here we use `request`, `session`, and `out`. Table [48] in Tutorialspoint lists all nine (including `application`, `config`, `pageContext`, `page`, `exception`). These implicit objects provide convenient access to application and HTTP context data without explicit lookup [18] [19].

Question 5(b) – Role-based Login (RBAC)

Q: *What is role-based login (Role-Based Access Control)? How can access be restricted using roles?*
A: Role-Based Access Control (RBAC) is an approach where each user is assigned one or more *roles*, and each role grants certain permissions [11]. Instead of assigning permissions directly to users, roles act as groups of permissions. For example, a `MANAGER` role might include permission to view reports. A user acquires those permissions by being in the `MANAGER` role.

To implement role-based login in a web app, one typically checks the user's role(s) at login and stores them (e.g. in a session). Then, before showing protected pages or actions, the application checks if the user's role permits that action. In Java, frameworks like Spring Security provide `<security:authorize>` tags in JSP to restrict content based on roles. For example:

```
<%@ taglib prefix="sec" uri="http://www.springframework.org/security/tags" %>
<sec:authorize access="hasRole('ROLE_USER')">
    <p>Welcome, authorized user!</p>
</sec:authorize>
```

This tag will only render the enclosed content if the user has `ROLE_USER`. Similarly, in servlets you might write:

```
HttpSession session = request.getSession(false);
String role = (String) session.getAttribute("role");
if ("ADMIN".equals(role)) {
    // allow access
} else {
    response.sendRedirect("accessDenied.jsp");
}
```

Thus, RBAC simplifies access control by grouping permissions into roles. As Wikipedia states, "roles are created for various job functions; permissions are assigned to roles; users are assigned to roles" [11]. In practice, frameworks enforce RBAC so that only users with the required role see certain links or pages, as demonstrated by the Spring Security tag example [20].

Question 5(c) – Session Management (short note)

Q: *Write a short note on session management in web applications.*
A: Session management refers to the techniques used to maintain state (user-specific data) across multiple HTTP requests in stateless web applications. Common methods include: - **HttpSession:** As shown earlier, `HttpSession` provides a server-side data store per user (session ID tracked via cookie). Attributes can be set and read. This is the standard Java EE approach [12] [15]. - **Cookies:** Small pieces of data sent to and from the client. Cookies can track sessions (e.g. `JSESSIONID`) or store persistent user info. - **URL rewriting:** Appending session ID to URLs if cookies are disabled. Effective session management is essential for login systems, shopping carts, etc. The servlet API (`HttpSession`) and JSP implicit session object provide built-in support, as previously illustrated [12] [15].

Question 5(d) – J2EE Architecture

Q: *Briefly describe the J2EE (Java EE) architecture.*
A: Java EE (formerly J2EE) architecture is a multi-tier model for enterprise applications. It typically includes: - **Client Tier:** Web browsers or standalone apps that interact with the application. - **Web Tier:** Servlets, JSPs, and web services running in a web container. Handles presentation and HTTP requests. - **Business Tier:** Enterprise JavaBeans (EJBs) or POJOs (e.g. Spring beans) that implement business logic. - **Enterprise Information System (EIS) Tier:** Databases and legacy systems. Accessed via JDBC, JPA/Hibernate, or JCA resource adapters.

These tiers communicate over standardized APIs. For example, a web request hits the web container (Tomcat/GlassFish), which may call EJBs in an EJB container or use JPA to access a database. Services like JNDI, JMS, and transactions integrate across tiers. The J2EE architecture diagram (described textually) shows a browser → web server (with servlet container) → application server (EJB container) → database. Components like servlets and EJBs are deployed in their respective containers.

In summary, J2EE is designed for scalable, portable, and robust enterprise applications with separation of concerns across tiers [5] [7].

This compilation covers representative questions from MCS-220 Web Technologies exams between June 2010 and June 2023. Each answer provides detailed explanations, examples, and citations to authoritative sources for deeper study.

---

[1] Web container - Wikipedia
https://en.wikipedia.org/wiki/Web_container

[2] [3] Life Cycle of a Servlet - GeeksforGeeks
https://www.geeksforgeeks.org/java/life-cycle-of-a-servlet/

[4] JSP Scripting Elements
https://www.w3schools.in/jsp/scripting-elements

[5] [7] 2. Introduction to Spring Framework
https://docs.spring.io/spring-framework/docs/4.0.x/spring-framework-reference/html/overview.html

[6] Dependency Injection :: Spring Framework
https://docs.spring.io/spring-framework/reference/core/beans/dependencies/factory-collaborators.html

[8] What Is Apache Struts | JRebel by Perforce
https://www.jrebel.com/blog/apache-struts

[9] [10] Struts 2 Value Stack and OGNL
https://www.tutorialspoint.com/struts_2/struts_value_stack_ognl.htm

[11] Role-based access control - Wikipedia
https://en.wikipedia.org/wiki/Role-based_access_control

[12] [15] HttpSession with example in Servlet
https://beginnersbook.com/2013/05/http-session/

[13] [14] JSP Life Cycle
https://www.tutorialspoint.com/jsp/jsp_life_cycle.htm

16 17 Servlet Collaboration In Java Using RequestDispatcher and HttpServletResponse - GeeksforGeeks
https://www.geeksforgeeks.org/java/servlet-collaboration-java-using-requestdispatcher-httpservletresponse/

18 19 JSP Implicit Objects
https://www.tutorialspoint.com/jsp/jsp_implicit_objects.htm

20 Spring Security Authentication and Authorization Using Jsp Pages
https://www.oodlestechnologies.com/blogs/spring-security-authentication-and-authorization-using-jsp-pages/